Experiment # 01

Introduction to Python and Basics Concepts

Objective

1. Setting up the development environment
2. Getting started with Python
3. To understand basics of Python programming language.

Software

- Anaconda i.e. Spyder

Theory

Steps are given below:

1. Download the setup using https://www.anaconda.com/
2. Go to your Downloads folder and double-click the installer to launch. To prevent permission errors, do not launch the installer from the Favorites folder.

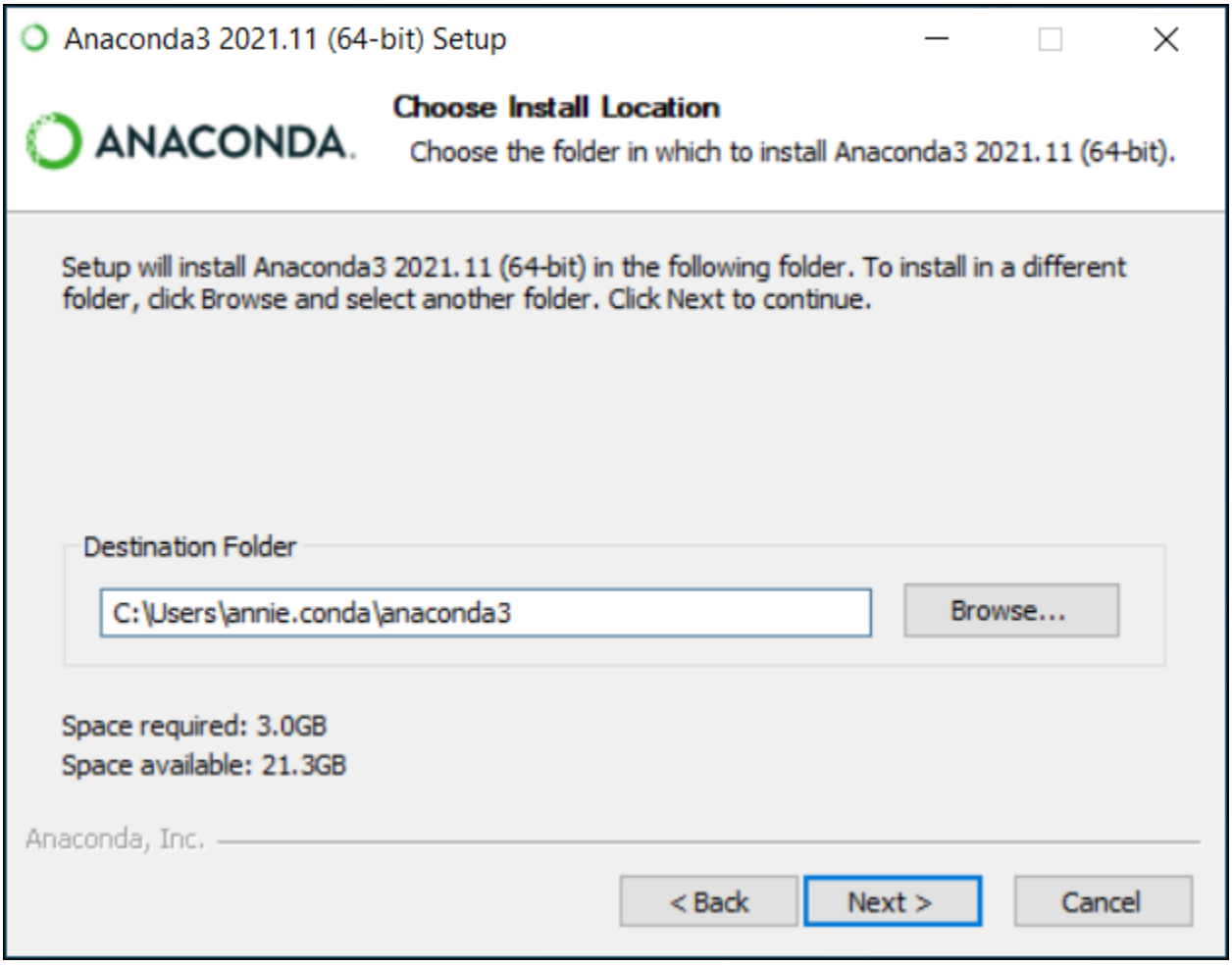
Note

If you encounter issues during installation, temporarily disable your anti-virus software during install, then re-enable it after the installation concludes. If you installed for all users, uninstall Anaconda and re-install it for your user only.

3. Click **Next**.
4. Read the licensing terms and click **I Agree**.
5. It is recommended that you install for **Just Me**, which will install Anaconda Distribution to just the current user account. Only select an install for **All Users** if you need to install for all users' accounts on the computer (which requires Windows Administrator privileges).
6. Click **Next**.
7. Select a destination folder to install Anaconda and click **Next**. Install Anaconda to a directory path that does not contain spaces or unicode characters. For more information on destination folders, see the FAQ.

Caution

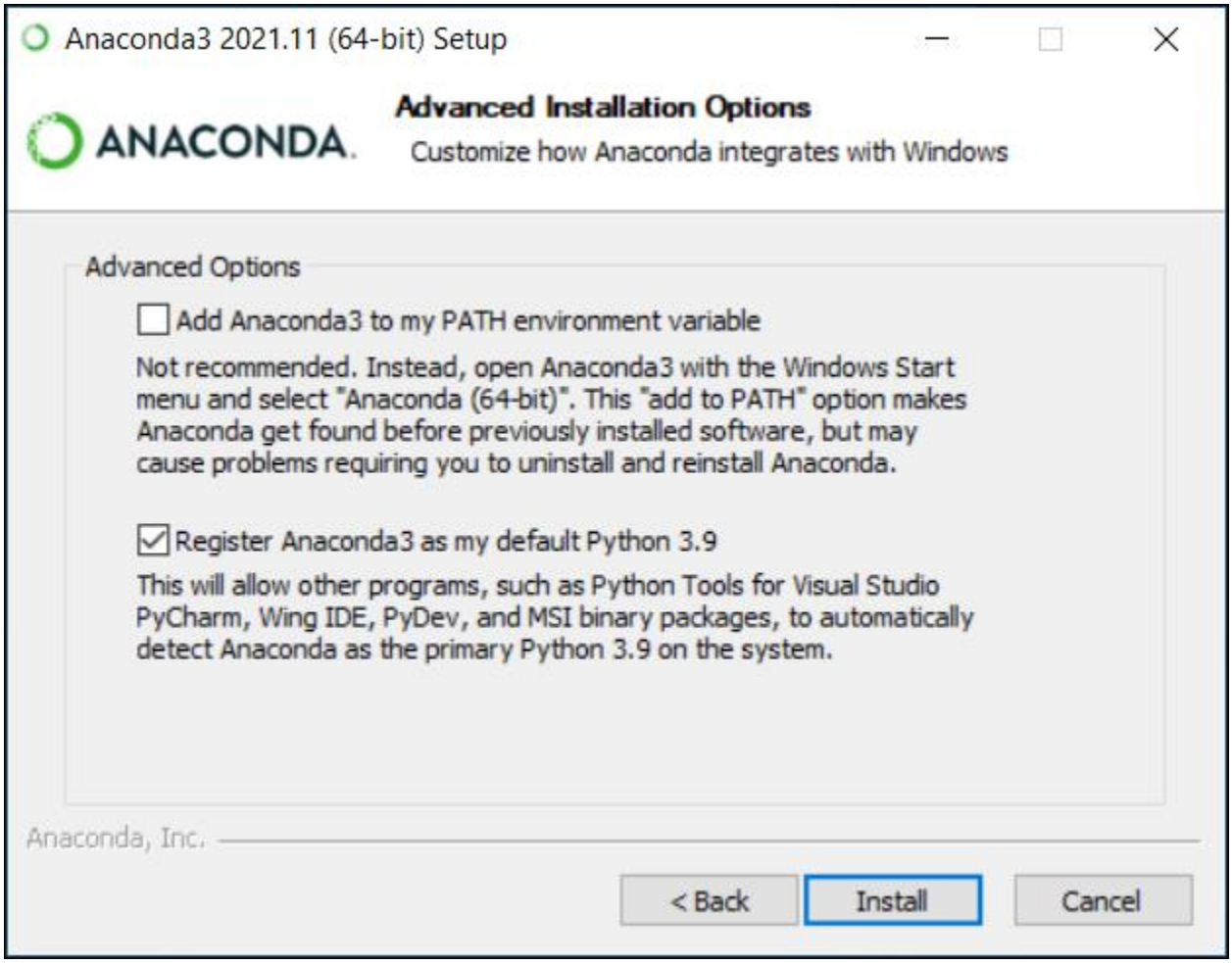
Do not install as Administrator unless admin privileges are required.



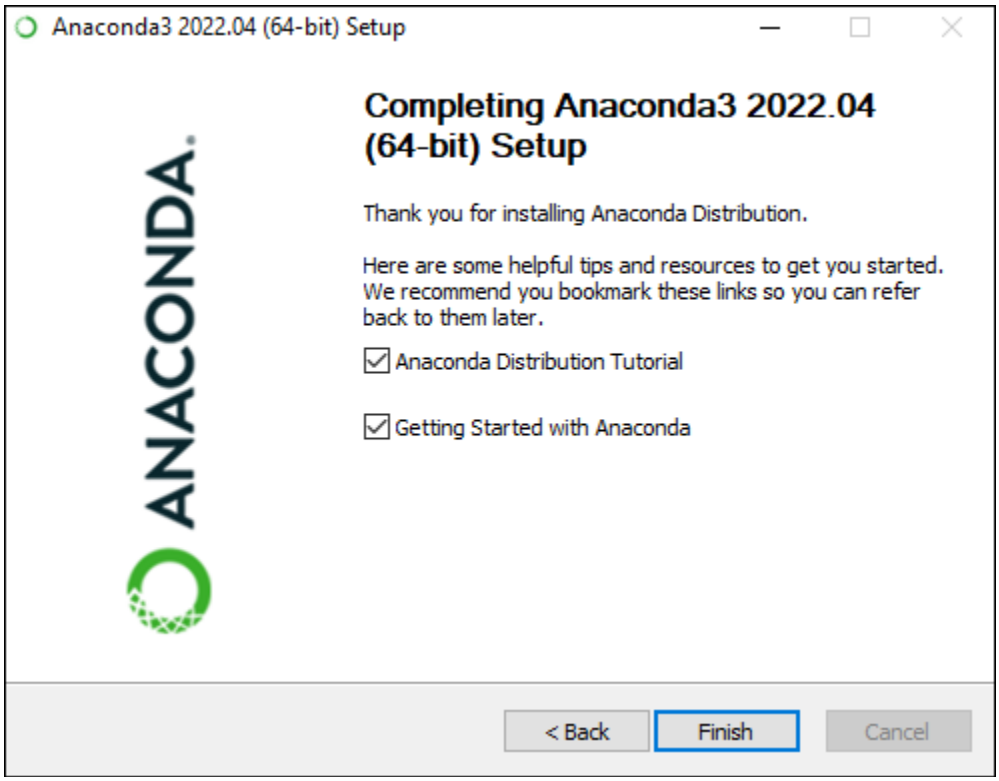
8. Choose whether to add Anaconda to your PATH environment variable or register Anaconda as your default Python. We **don't recommend** adding Anaconda to your PATH environment variable, since this can interfere with other software. Unless you plan on installing and running multiple versions of Anaconda or multiple versions of Python, accept the default and leave this box checked. Instead, use Anaconda software by opening Anaconda Navigator or the Anaconda Prompt from the Start Menu.

Note

As of Anaconda Distribution 2022.05, the option to add Anaconda to the PATH environment variable during an **All Users** installation has been disabled. This was done to address a security exploit. You can still add Anaconda to the PATH environment variable during a **Just Me** installation.



9. Click Install. If you want to watch the packages Anaconda is installing, click Show Details.
10. Click Next.
11. After a successful installation you will see the “Thanks for installing Anaconda” dialog box:



12. If you wish to read more about Anaconda.org and how to get started with Anaconda, check the boxes "Anaconda Distribution Tutorial" and "Learn more about Anaconda". Click the Finish button.

Confirmation of installation
Confirm that Anaconda is installed and working with Anaconda Navigator or conda with the following instructions.

- **Anaconda Navigator**

Anaconda Navigator is a graphical user interface (GUI) that is automatically installed with Anaconda. Navigator will open if the installation was successful. If Navigator does not open, review our help resources.

Windows: Click ***Start***, search for Anaconda Navigator, and click to open.
macOS: Click ***Launchpad*** and select Anaconda Navigator. Or use ***Cmd+Space*** to open Spotlight Search and type "***Navigator***" to open the program.
Linux: See next section.

Conda
If you prefer using a command line interface (CLI), use conda to verify the installation using Anaconda Prompt on Windows or the terminal on Linux and macOS.

To open Anaconda Prompt:

Windows: Click ***Start,*** search for Anaconda Prompt, and click to open.
macOS: Use ***Cmd+Space*** to open Spotlight Search and type "Navigator" to open the program.
Linux–CentOS: ***Open Applications > System Tools > terminal***.
Linux–Ubuntu: Open the Dash by clicking the Ubuntu icon, then type "***terminal***".
After opening Anaconda Prompt or the terminal, choose any of the following methods to verify:

- Enter ***conda list***. If Anaconda is installed and working, this will display a list of installed packages and their versions.
- Enter the command ***python***. This command runs the Python shell, also known as the REPL. If Anaconda is installed and working, the version information it displays when it starts up will include "Anaconda". To exit the Python shell, enter the command ***quit().***
- Open Anaconda Navigator with the command ***anaconda-navigator***. If Anaconda is installed properly, Anaconda Navigator will open.

3. Python Basics

Python Features
Python's features include:
Easy-to-learn: Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.

Easy-to-read: Python code is more clearly defined and visible to the eyes.

Easy-to-maintain: Python's source code is fairly easy-to-maintain.

A broad standard library: Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.

Interactive Mode: Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.

Portable: Python can run on a wide variety of hardware platforms and has the same interface on all platforms.

Extendable: You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.

Databases: Python provides interfaces to all major commercial databases.

GUI Programming: Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.

Scalable: Python provides a better structure and support for large programs than shell scripting.

Apart from the above-mentioned features, Python has a big list of good features, few are listed below:

- It supports functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.

Python Variables

Variable is a name which is used to refer memory location. Variable also known as identifier and used to hold value.

In Python, we don't need to specify the type of variable because Python is a type infer language and smart enough to get variable type.

Variable names can be a group of both letters and digits, but they have to begin with a letter or an underscore.

It is recommended to use lowercase letters for variable name. Rahul and rahul both are two different variables.

Identifier Naming

Variables are the example of identifiers. An Identifier is used to identify the literals used in the program.

The rules to name an identifier are given below.

The first character of the variable must be an alphabet or underscore (_).

All the characters except the first character may be an alphabet of lower-case(a-z), upper-case (A-Z), underscore or digit (0-9).

Identifier name must not contain any white-space, or special character (!, @, #, %, ^, &, *).

Identifier name must not be like any keyword defined in the language.

Identifier names are case sensitive, for example my name, and MyName is not the same.

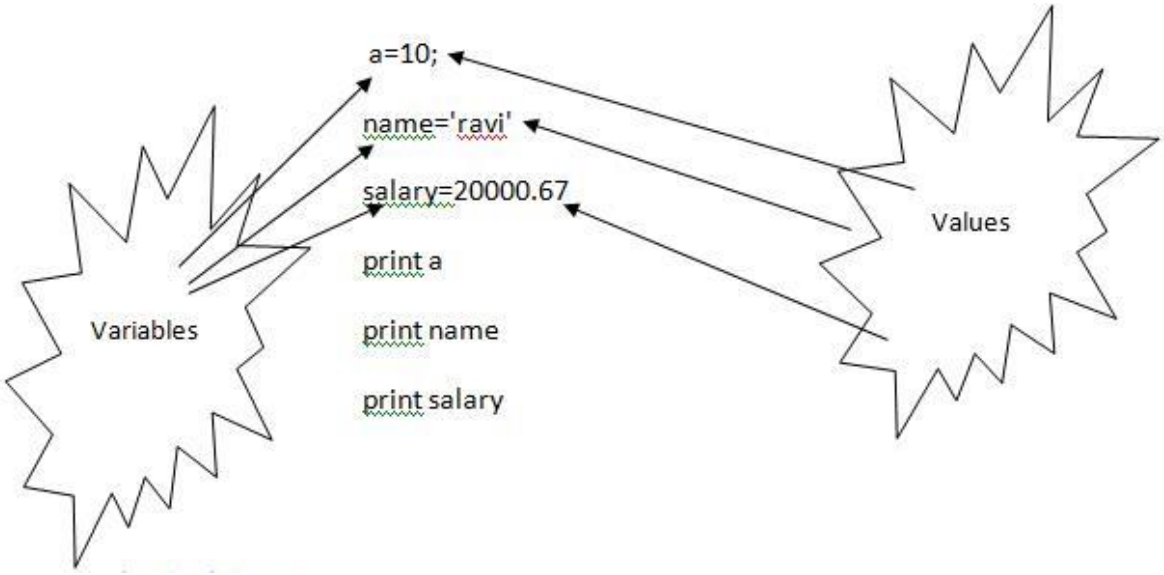
Examples of valid identifiers: a123, _n, n_9, etc.
Examples of invalid identifiers: 1a, n%4, n 9, etc.

Declaring Variable and Assigning Values

Python does not bind us to declare variables before using in the application. It allows us to create variables at the required time.

We don't need to declare explicitly variables in Python. When we assign any value to the variable that variable is declared automatically.

The equal (=) operator is used to assign value to a variable.



Multiple Assignment

Python allows us to assign a value to multiple variables in a single statement which is also known as multiple assignment.

We can apply multiple assignments in two ways either by assigning a single value to multiple variables or assigning multiple values to multiple variables. Let’s see given examples.

1. Assigning single value to multiple variables

Eg:

```
X = y = z = 50
print x
print y
print z
```

2.Assigning multiple values to multiple variables:

Eg:

```
a , b , c = 5 , 10 , 15
print a
print b
print c
```

The values will be assigned in the order in which variables appear.

Python Data Types

Variables can hold values of different data types. Python is a dynamically typed language hence we need not define the type of the variable while declaring it. The interpreter implicitly binds the value with its type.

Python enables us to check the type of variable used in the program. Python provides us the **type()** function which returns the type of the variable passed.

Consider the following example to define the values of different data types and check its type.

```
a=10
b="Hi Python"
c = 10.5
print(type(a));
print(type(b));
print(type(c));
```

Standard data types

A variable can hold different types of values. For example, a person's name must be stored as a string whereas its id must be stored as an integer.

Python provides various standard data types that define the storage method on each of them. The data types defined in Python are given below.

1. Numbers
2. String
3. List
4. Tuple
5. Dictionary

In this section of the tutorial, we will give a brief introduction of the above data types. We will discuss each one of them in detail later in this tutorial.

Numbers

Number stores numeric values. Python creates Number objects when a number is assigned to a variable. For example;

```
a = 3 , b = 5 #a and b are number objects
```

Python supports 4 types of numeric data.

1. int (signed integers like 10, 2, 29, etc.)
2. long (long integers used for a higher range of values like 908090800L, -0x1929292L, etc.)
3. float (float is used to store floating point numbers like 1.9, 9.902, 15.2, etc.)
4. complex (complex numbers like 2.14j, 2.0 + 2.3j, etc.)

Python allows us to use a lower-case L to be used with long integers. However, we must always use an upper-case L to avoid confusion.

A complex number contains an ordered pair, i.e., x + iy where x and y denote the real and imaginary parts respectively.

String:

The string can be defined as the sequence of characters represented in the quotation marks. In python, we can use single, double, or triple quotes to define a string.

String handling in python is a straightforward task since there are various inbuilt functions and operators provided.

In the case of string handling, the operator + is used to concatenate two strings as the operation *"hello"+" python"* returns *"hello python"*.

The operator * is known as repetition operator as the operation "Python " *2 returns "Python Python".

The following example illustrates the string handling in python.

1. str1 = 'hello javatpoint' #string str1
2. str2 = ' how are you' #string str2
3. **print** (str1[0:2]) #printing first two character using slice operator
4. **print** (str1[4]) #printing 4th character of the string
5. **print** (str1 + str2) #printing the concatenation of str1 and str2

Python Keywords

Python Keywords are special reserved words which convey a special meaning to the compiler/interpreter. Each keyword has a special meaning and a specific operation. These keywords can't be used as variable. Following is the List of Python Keywords.

True	False	none	and	as
Asset	Def	class	continue	break
Else	finally	elif	del	except
global	For	if	from	import
raise	Try	or	return	pass
nonlocal	In	not	is	lambda

Operators

Arithmetic operators

Arithmetic operators are used to perform arithmetic operations between two operands. It includes +(addition), - (subtraction), *(multiplication), /(divide), %(reminder), //(floor division), and exponent (**).

Consider the following table for a detailed explanation of arithmetic operators.

Operator	Description
+ (Addition)	It is used to add two operands. For example, if a = 20, b = 10 => a+b = 30
- (Subtraction)	It is used to subtract the second operand from the first operand. If the first operand is less than the second operand, the value result negative. For example, if a = 20, b = 10 => a - b = 10
/ (divide)	It returns the quotient after dividing the first operand by the second operand. For example, if a = 20, b = 10 => a/b = 2
*(Multiplication)	It is used to multiply one operand with the other. For example, if a = 20, b = 10 => a * b = 200
% (reminder)	It returns the reminder after dividing the first operand by the second operand. For example, if a = 20, b = 10 => a%b = 0
** (Exponent)	It is an exponent operator represented as it calculates the first operand power to second operand.
// (Floor division)	It gives the floor value of the quotient produced by dividing the two operands.

Comparison operator

Comparison operators are used to comparing the value of the two operands and returns Boolean true or false accordingly. The comparison operators are described in the following table.

Operator	Description
==	If the value of two operands is equal, then the condition becomes true.
!=	If the value of two operands is not equal then the condition becomes true.
<=	If the first operand is less than or equal to the second operand, then the condition becomes true.
>=	If the first operand is greater than or equal to the second operand, then the condition becomes true.
<>	If the value of two operands is not equal, then the condition becomes true.
>	If the first operand is greater than the second operand, then the condition becomes true.
<	If the first operand is less than the second operand, then the condition becomes true.

Membership Operators

Python membership operators are used to check the membership of value inside a data structure. If the value is present in the data structure, then the resulting value is true otherwise it returns false.

Operator	Description
in	It is evaluated to be true if the first operand is found in the second operand (list, tuple, or dictionary
not in	It is evaluated to be true if the first operand is not found in the second operand (list, tuple, or dictionary).

Python If-else statements

Decision making is the most important aspect of almost all the programming languages. As the name implies, decision making allows us to run a particular block of code for a particular decision. Here, the decisions are made on the validity of the particular conditions. Condition checking is the backbone of decision making.

In python, decision making is performed by the following statements.

Statement	Description
If Statement	The if statement is used to test a specific condition. If the condition is true, a block of code (if-block) will be executed.
If - else Statement	The if-else statement is similar to if statement except the fact that, it also provides the block of the code for the false case of the condition to be checked. If the condition provided in the if statement is false, then the else statement will be executed.
Nested if Statement	Nested if statements enable us to use if ? else statement inside an outer if statement.

1. **The syntax of the if-statement is given below.**

```
if expression:
   statement
```

2. **The syntax of the if-else statement is given below.**

```
if condition:
   #block of statements
else:
   #another block of statements (else-block)
```

3. **The syntax of the elif statement**

```
 if expression 1:
# block of statements
  elif expression 2:
 # block of statements
 elif expression 3:
```

```
  # block of statements
  else:
  # block of statements
```

Python for loop

The 'for' loop in Python is used to iterate the statements or a part of the program several times. It is frequently used to traverse the data structures like list, tuple, or dictionary.

The syntax of for loop in python is given below.

```
for iterating_var in sequence:
  statement(s)
```

Nested for loop in python

Python allows us to nest any number of for loops inside a for loop. The inner loop is executed n number of times for every iteration of the outer loop.

The syntax of the nested for loop in python is given below.

```
for iterating_var1 in sequence:
  for iterating_var2 in sequence:
    #block of statements
#Other statements
```

Lab Tasks

1. Run all the above program that are given in the examples.
2. Write a Python code to solve the quadratic equation. (hint: import cmath library and use cmath.sqrt for this question)
3. Write a Python code that implements basic calculator functioning (+, -, *, /) for user provided data of below mentioned data types.
   a. int
   b. float